

# Comportamento Temporal

## Média

Representa o **valor central do sinal** ou a sua componente contínua (DC).

$$\mu = \frac{1}{N} \sum_{n=0}^{N-1} x[n]$$

Significado: indica o nível de deslocamento (offset) do sinal em relação ao zero.

## Variância

Mede a **dispersão do sinal em relação à sua média**.

$$\sigma^2 = \frac{1}{N} \sum_{n=0}^{N-1} (x[n] - \mu)^2$$

Significado: Quantifica o quanto o sinal oscila ao redor da média. Quanto maior a variância, mais ruidoso ou dinâmico é o sinal.

## Valor Eficaz / RMS (Root Mean Square)

Raiz quadrada da média dos quadrados do sinal. Corresponde ao valor de tensão ou corrente contínua equivalente que produziria a mesma dissipação de potência em uma carga resistiva.

$$x_{\text{RMS}} = \frac{1}{N} \sum_{n=0}^{N-1} x[n]^2$$

Relação com a média e variância:

$$x_{\text{RMS}}^2 = \mu^2 + \sigma^2$$

(Potência total = Potência DC + Potência AC)

## Autocorrelação ( $R_{xx}[k]$ )

Mede o **grau de similaridade entre o sinal  $x[n]$  e uma versão deslocada no tempo de si mesmo por  $k$  amostras**.

$$R_{xx}[k] = \sum_n x[n] \cdot x[n+k]$$

Significado: Revela frequências fundamentais e ecos escondidos no sinal, mesmo na presença de ruído.

## Domínio da Frequência

### FFT (Fast Fourier Transform)

Transforma todo o sinal do tempo diretamente para o domínio da frequência, decompondo em uma soma de senos e cossenos.

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j \frac{2\pi}{N} kn}$$

A limitação é que se perde a informação temporal.

## STFT (Short-Time Fourier Transform)

Para contornar a limitação da FFT em sinais não estacionários (como voz ou música), a STFT aplica a FFT em pequenas janelas de tempo deslizando sobre o sinal.

$$X(t, f) = \int x(\tau)w(\tau - t)e^{-j2\pi f\tau}d\tau$$

Basicamente, divide-se o sinal em blocos e calcula a FFT para cada bloco. Como resultado, gera-se um **espectograma** (tempo vs frequência vs amplitude).

Janelas curtas dão boa resolução temporal, mas baixa resolução em frequência. Janelas longas dão o oposto.

## Wavelets (Transformada Wavelet)

Em vez de usar senos e cossenos infinitos como a Fourier, a transformada Wavelet decompõe o sinal usando “pequenas ondas” (wavelets) com duração e frequência limitadas.

$$WT(a, b) = \frac{1}{\sqrt{|a|}} \int x(t)\varphi^*\left(\frac{t-b}{a}\right)dt$$

Utiliza janelas de tamanho variável:

- Frequências altas: Janelas temporais estreitas (alta resolução no tempo)
- Frequências baixas: Janelas temporais largas (alta resolução em frequência)

Resolve o dilema da STFT, adaptando a resolução conforme a faixa de frequência.

## PSD (Power Spectral Density)

Indica como a potência do sinal está distribuída ao longo das frequências.

$$P_{xx}(f) = \lim_{T \rightarrow \infty} \frac{1}{T} |X_T(f)|^2$$

É a transformada de Fourier da função de autocorrelação do sinal (Teorema de Wiener-Khinchin).

## Filtragem

### Low-Pass

Deixa passar frequências abaixo de uma frequência de corte e atenua as frequências acima dela.

Aplicação: Remover ruído elétrico agudo em gravações de voz ou suavizar leituras de sensores (como acelerômetros).

### High-Pass

Deixa passar frequências acima de uma frequência de corte e atenua as frequências abaixo dela.

Aplicação: Eliminar frequência DC da alimentação.

### Notch

Atenua fortemente uma faixa muito estreita de frequências ao redor de uma frequência central, deixando passar todo o resto.

Aplicação: clássico filtro Notch de 60Hz para remover o hum elétrico da tomada captado por sensores.

## Passa-Faixa (Band-Pass)

Deixa passar apenas as frequências contidas entre duas frequências de corte.

Aplicação: Sintonia de rádio ou isolar ritmos específicos de um sinal de EEG (ex: ondas Alfa entre 8Hz e 12Hz)

## Exemplo em Python

Vamos usar as bibliotecas: `pip install numpy scipy pywavelets matplotlib`.

### Código

```
import numpy as np
import scipy.signal as signal
import scipy.fft as fft
import pywt
import matplotlib.pyplot as plt

# =====
# 0. CRIANDO SINAL DISCRETO DE TESTE
# =====
fs = 1000 # Frequência de amostragem (1000 Hz)
t = np.linspace(0, 1.0, fs, endpoint=False) # 1 segundo (1000 amostras)

# Componentes do sinal
sinal_limpo = 3 * np.sin(2 * np.pi * 10 * t) # 10 Hz
ruído_60hz = 1.5 * np.sin(2 * np.pi * 60 * t) # 60 Hz (interferência)
ruído_branco = np.random.normal(0, 0.5, size=fs) # Ruído aleatório
offset_dc = 2.0 # Componente DC

sinal = offset_dc + sinal_limpo + ruído_60hz + ruído_branco

# =====
# 1. ANÁLISE NO TEMPO (VALORES ÚNICOS)
# =====
media = np.mean(sinal)
variancia = np.var(sinal)
desvio_padrao = np.std(sinal)
rms = np.sqrt(np.mean(sinal**2))
energia_total = np.sum(sinal**2)
potencia_media = energia_total / len(sinal)

# Autocorrelação
autocorr = signal.correlate(sinal, sinal, mode='full')
lags = signal.correlation_lags(len(sinal), len(sinal))
rxx_zero = autocorr[len(sinal) - 1] # Rxx[0] equivale à Energia Total

print("="*50)
print(" 1. MÉTRICAS ÚNICAS NO DOMÍNIO DO TEMPO")
print("="*50)
print(f"• Média (Componente DC) : {media:.4f}")
print(f"• Variância (Potência AC) : {variancia:.4f}")
print(f"• Desvio Padrão : {desvio_padrao:.4f}")
print(f"• RMS (Valor Eficaz) : {rms:.4f}")
print(f"• Potência Média (RMS²) : {potencia_media:.4f}")
print(f"• Energia Total do Sinal : {energia_total:.4f}")
print(f"• Autocorrelação em Lag Zero Rxx[0]: {rxx_zero:.4f}\n")
```

```

# =====
# 2. ANÁLISE EM FREQUÊNCIA (MÉTRICAS ÚNICAS)
# =====
# FFT
fft_valores = fft.fft(sinal)
fft_amp = np.abs(fft_valores)[:fs // 2] * (2 / fs)
freqs = fft.fftfreq(len(sinal), 1/fs)[:fs // 2]
pico_freq_fft = freqs[np.argmax(fft_amp[1:])] + 1 # Frequência dominante (ignorando DC)

# STFT
f_stft, t_stft, Zxx = signal.stft(sinal, fs=fs, nperseg=128)

# Wavelets (CWT)
coefs, freqs_wavelet = pywt.cwt(sinal, scales=np.arange(1, 128), wavelet='morl',
sampling_period=1/fs)
energia_wavelet_total = np.sum(np.abs(coefs)**2)

# PSD (Welch)
f_psd, psd = signal.welch(sinal, fs=fs, nperseg=256)
potencia_psd_integrada = np.trapz(psd, f_psd) # Área sob a curva da PSD

print("="*50)
print(" 2. MÉTRICAS ÚNICAS NO DOMÍNIO DA FREQUÊNCIA")
print("="*50)
print(f"• Frequência Dominante (FFT)           : {pico_freq_fft:.2f} Hz")
print(f"• Maior Amplitude Espectral (FFT)        : {np.max(fft_amp[1:]):.4f}")
print(f"• Potência Integrada da PSD                : {potencia_psd_integrada:.4f} V²")
print(f"• Energia Total nos Coefs Wavelet         : {energia_wavelet_total:.4f}\n")

# =====
# 3. FILTRAGEM (MÉTRICAS COMPARATIVAS ÚNICAS)
# =====
# Passa-Baixa (< 20 Hz)
b_lp, a_lp = signal.butter(4, 20, btype='lowpass', fs=fs)
sinal_lp = signal.filtfilt(b_lp, a_lp, sinal)

# Passa-Alta (> 5 Hz)
b_hp, a_hp = signal.butter(4, 5, btype='highpass', fs=fs)
sinal_hp = signal.filtfilt(b_hp, a_hp, sinal)

# Notch (Remove 60 Hz)
b_notch, a_notch = signal.iirnotch(w0=60, Q=30, fs=fs)
sinal_notch = signal.filtfilt(b_notch, a_notch, sinal)

# Passa-Faixa (8 - 12 Hz)
b_bp, a_bp = signal.butter(4, [8, 12], btype='bandpass', fs=fs)
sinal_bp = signal.filtfilt(b_bp, a_bp, sinal)

print("="*50)
print(" 3. IMPACTO DOS FILTROS (MÉDIAS E RMS PÓS-FILTRO)")
print("="*50)
print(f"• Passa-Baixa (<20Hz)    -> Média: {np.mean(sinal_lp):.4f} | RMS: {np.sqrt(np.mean(sinal_lp**2)):.4f}")
print(f"• Passa-Alta (>5Hz)      -> Média: {np.mean(sinal_hp):.4f} | RMS: ")

```

```

{np.sqrt(np.mean(sinal_hp**2)):.4f} (DC Removido)")
print(f"• Notch (60Hz) -> Média: {np.mean(sinal_notch):.4f} | RMS: {np.sqrt(np.mean(sinal_notch**2)):.4f}")
print(f"• Passa-Faixa (8-12Hz) -> Média: {np.mean(sinal_bp):.4f} | RMS: {np.sqrt(np.mean(sinal_bp**2)):.4f}\n")

# =====
# 4. PLOT DOS GRÁFICOS
# =====
plt.figure(figsize=(15, 12))

# Tempo
plt.subplot(3, 3, 1)
plt.plot(t, sinal, color='black', alpha=0.7)
plt.axhline(media, color='red', linestyle='--', label=f'Média={media:.2f}')
plt.title('1. Sinal com Ruído'); plt.xlabel('Tempo (s)'); plt.legend()

plt.subplot(3, 3, 2)
plt.plot(lags / fs, autocorr, color='purple')
plt.title('2. Autocorrelação'); plt.xlabel('Lag (s)')

# Frequência
plt.subplot(3, 3, 3)
plt.plot(freqs, fft_amp, color='blue')
plt.xlim(0, 100); plt.title('3. FFT (Amplitude)'); plt.xlabel('Frequência (Hz)')

plt.subplot(3, 3, 4)
plt.pcolormesh(t_stft, f_stft, np.abs(Zxx), shading='gouraud', cmap='viridis')
plt.ylim(0, 100); plt.title('4. STFT (Espectrograma)'); plt.xlabel('Tempo (s)')

plt.subplot(3, 3, 5)
plt.pcolormesh(t, freqs_wavelet, np.abs(coefs), shading='gouraud', cmap='magma')
plt.ylim(0, 100); plt.title('5. Wavelet CWT'); plt.xlabel('Tempo (s)')

plt.subplot(3, 3, 6)
plt.semilogy(f_psd, psd, color='green')
plt.xlim(0, 100); plt.title('6. PSD (Welch)'); plt.xlabel('Frequência (Hz)')

# Filtragem
plt.subplot(3, 3, 7)
plt.plot(t, sinal, color='gray', alpha=0.3)
plt.plot(t, sinal_lp, color='crimson', label=f'Passa-Baixa (RMS={np.sqrt(np.mean(sinal_lp**2)):.2f})')
plt.title('7. Passa-Baixa'); plt.legend()

plt.subplot(3, 3, 8)
plt.plot(t, sinal, color='gray', alpha=0.3)
plt.plot(t, sinal_notch, color='darkorange', label=f'Notch 60Hz (RMS={np.sqrt(np.mean(sinal_notch**2)):.2f})')
plt.title('8. Filtro Notch'); plt.legend()

plt.subplot(3, 3, 9)
plt.plot(t, sinal_limpo, color='black', linestyle=':', label='Ideal 10Hz')
plt.plot(t, sinal_bp, color='teal', label=f'Passa-Faixa (RMS={np.sqrt(np.mean(sinal_bp**2)):.2f})')
plt.title('9. Passa-Faixa (Isolado)'); plt.legend()

```

```
plt.tight_layout()
plt.savefig('graph.png', dpi=300, bbox_inches='tight')
plt.show()
```

## Output do Código (Texto + Gráfico)

### 1. MÉTRICAS ÚNICAS NO DOMÍNIO DO TEMPO

- Média (Componente DC) : 1.9986
- Variância (Potência AC) : 5.8764
- Desvio Padrão : 2.4241
- RMS (Valor Eficaz) : 3.1418
- Potência Média ( $\text{RMS}^2$ ) : 9.8708
- Energia Total do Sinal : 9870.7548
- Autocorrelação em Lag Zero  $\text{Rxx}[0]$ : 9870.7548

### 2. MÉTRICAS ÚNICAS NO DOMÍNIO DA FREQUÊNCIA

- Frequência Dominante (FFT) : 10.00 Hz
- Maior Amplitude Espectral (FFT) : 2.9968
- Potência Integrada da PSD : 5.8030  $\text{V}^2$
- Energia Total nos Coefs Wavelet : 15479790.4789

### 3. IMPACTO DOS FILTROS (MÉDIAS E RMS PÓS-FILTRO)

- Passa-Baixa (<20Hz) -> Média: 1.9948 | RMS: 2.9045
- Passa-Alta (>5Hz) -> Média: 0.0359 | RMS: 2.4003 (DC Removido)
- Notch (60Hz) -> Média: 2.0004 | RMS: 2.9615
- Passa-Faixa (8-12Hz) -> Média: 0.0293 | RMS: 1.8637

